Project 9 — Security Architecture Document

Deliverable 5. One section per control layer, each naming the specific file and function and the threat it mitigates.

Section 1 — Authentication

- **File / function:** `app.py` → `token_required` decorator (wraps `chat`).
- **Control:** reads the JWT from the `Authorization: Bearer` header, verifies it with `jwt.decode` using a secret loaded only from the `JWT_SECRET` environment variable; returns **401** for missing, malformed, expired, or invalid tokens. The model is never called until verification passes.
- **Threat mitigated:** unauthenticated access / broken access control — stops anonymous callers from reaching the AI or consuming the model.

Section 2 — Input Validation

- **File / function:** `validate.py` → `validate_input`, called in `app.py` `chat()` before the model.
- **Control:** Layer 1 rejects input > 500 chars; Layer 2 blocks injection phrases ("ignore previous instructions", "you are now", "pretend you are", "act as if you have no") and 40+ char base64 strings. Returns **400**.
- **Threat mitigated:** prompt injection / jailbreak (OWASP LLM01), oversized- input abuse.

Section 3 — Output Scanning

- **File / function:** `output_scanner.py` → `scan_output`, called in `app.py` `chat()` before returning the response.
- **Control:** blocks responses containing verbatim system-prompt markers, credential patterns (`password=`, `api_key=`, `token=`, `sk-…`), or unsolicited URLs, replacing them with a safe error message.
- **Threat mitigated:** sensitive information disclosure / system-prompt leakage (OWASP LLM06, LLM10).

Section 4 — Dependency Security

- **File:** `requirements.txt` (baseline) vs `requirements_pinned.txt` (hardened), audited via `pip-audit` (see `PIP_AUDIT_COMPARISON.md`).
- **Control:** exact version pins to a known non-vulnerable set; Critical/High CVEs upgraded to the minimum safe version.
- **Threat mitigated:** supply-chain / vulnerable-dependency exploitation (OWASP LLM05).

Section 5 — CI/CD Pipeline

- **File / function:** `.github/workflows/security.yml` (4 gates) and `inject_scanner.py` → `main`.
- **Control:** every push runs Bandit (SAST), Safety (dependency CVEs), Gitleaks (secrets), and the custom injection scanner; all 4 must pass before a commit is deployable.
- **Threat mitigated:** regressions — hardcoded secrets, vulnerable deps, dangerous code, and hardcoded prompts cannot reach production.

Data flow (threat model)

`User → JWT auth (401) → Flask /chat → Input validation (400) → Ollama → Output scanner (safe error) → Response`. Trust boundary sits at the JWT auth step; every downstream control treats input and model output as untrusted. See `threat_model.drawio`.